

fid-synth___fidelity-binomial-f-k4

An AgentMPI run analysis

Abstract

This is the surrogate-executor companion to the real binomial fidelity run, taken at $p = 16$ six minutes after the commit that introduced the k -ary algorithm, and it is a plumbing check rather than evidence about operators: `executors` records `function` $\times 16$, so the merge kernel is `_surrogate_merge(0.08, 0)`, a Python function whose docstring says outright that it reproduces the paper’s loss model rather than testing it. The structural measurements it does establish are exact — `fold_depth:` $4 = \lceil \log_2 16 \rceil$, 4 rounds, 15 messages = $p - 1$, 15 operator applications, model check passing — and the retention figure of 0.7031 should be read only as a statement about the surrogate. It is worth reading, because the surrogate does not behave as documented. Reconstructing the fold from the message blobs shows that what it drops is not a fact but an entire input: `json.dumps` renders each report as one line, the kernel splits the prompt by lines, and so the “items” it draws over are the whole left input, the whole right input, and one line of the prompt’s own instructions — “such as [F-3-2]. Keep every identifier that appears in either input, exactly as” — which is present verbatim in the stored accumulators. Two consequences follow and both matter for the campaign. The phantom [F-3-2] is scored as a retained item, which is exactly why rank 3 reads 0.25 here while its real report was destroyed at the first merge. And because the kernel reseeds from `seed + len(prompt)`, five structurally identical depth-1 merges with byte-identical prompt lengths made byte-identical draws and all five dropped their odd-numbered partner: ranks 1, 3, 5, 7 and 9, exactly the observed zeros. Loss here is a deterministic function of prompt length, not an independent per-item event, so this run cannot support the $(1 - r)^d$ curve it was built to exhibit.

1 What this run is

property	value
run	fid-synth___fidelity-binomial-f-k4
experiment	-
campaign label	-
declared world size	16
ranks appearing in the log	16
executors	<code>function</code> $\times 16$
events	95
wall time	0.10 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.04×	total busy time over wall time
parallel efficiency	0.2%	achieved parallelism per rank
idle fraction	99.8%	rank-seconds paid for and unused
peak ranks busy	1	of 16 in the world
serial fraction of active time	100.0%	time with exactly one rank busy
load imbalance	12.00×	slowest rank's busy time over the mean
coordination share	26.4%	rank-seconds blocked in collectives, of those available
coordination in flight	97.5%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	15	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	15	15 eager, 0 rendezvous
tokens sent	5,416	0 deferred under rendezvous
tokens in / out	11,762 / 11,833	prompt and completion
cost	\$0.2128	0.0180 \$/kTok out

3 What the analysis notices

- **[note]** Occupancy is not measurable for this run: its executors (function) emit no broker claim events, so busy time, achieved parallelism, and the idle fraction are artefacts of the instrumentation rather than properties of the run.
- **[note]** Load imbalance is 12.00×: the slowest rank was busy far longer than the mean.
- **[ok]** All 1 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.
- **[ok]** All 1 checkable collective(s) also matched the predicted round count.

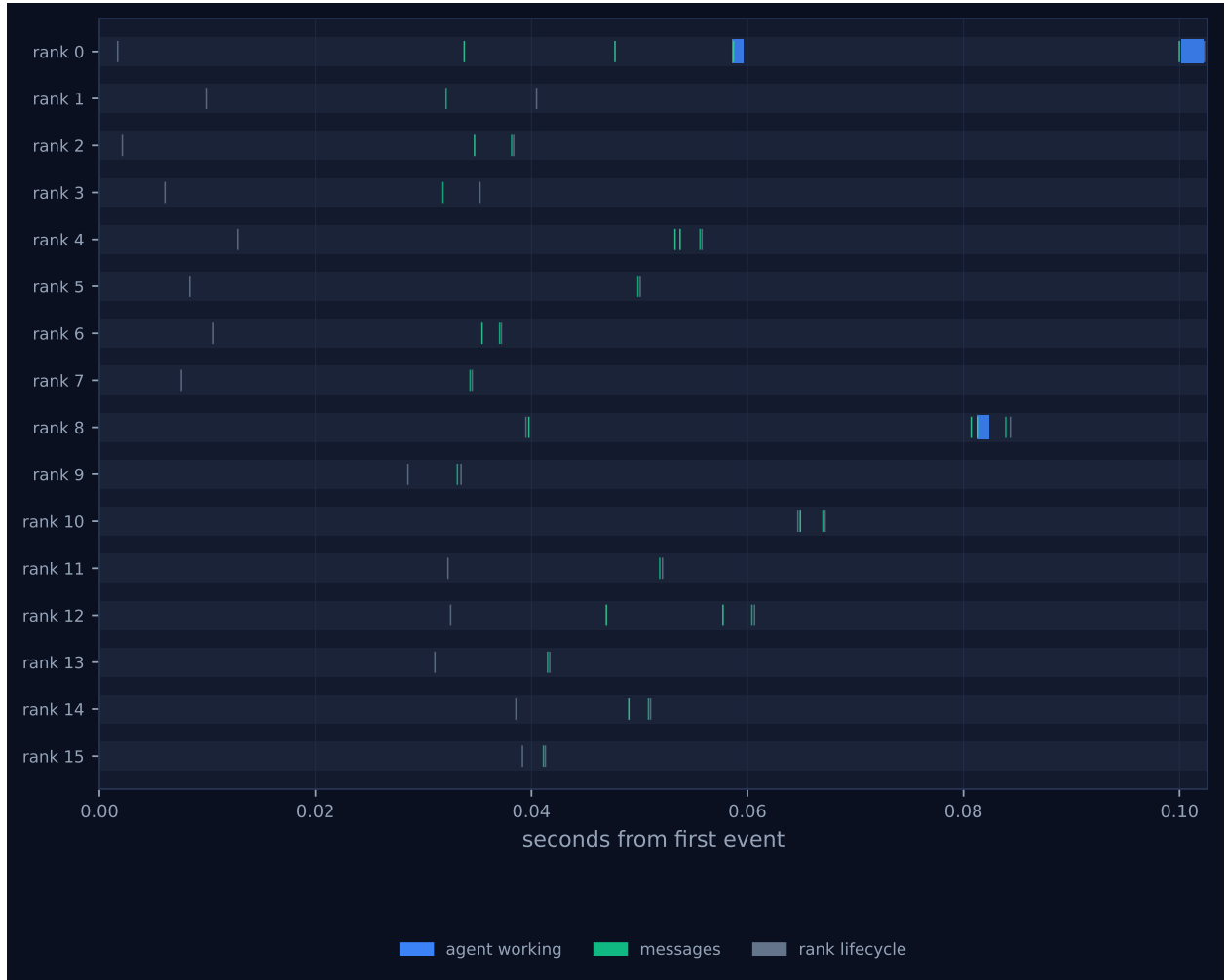


Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.



Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

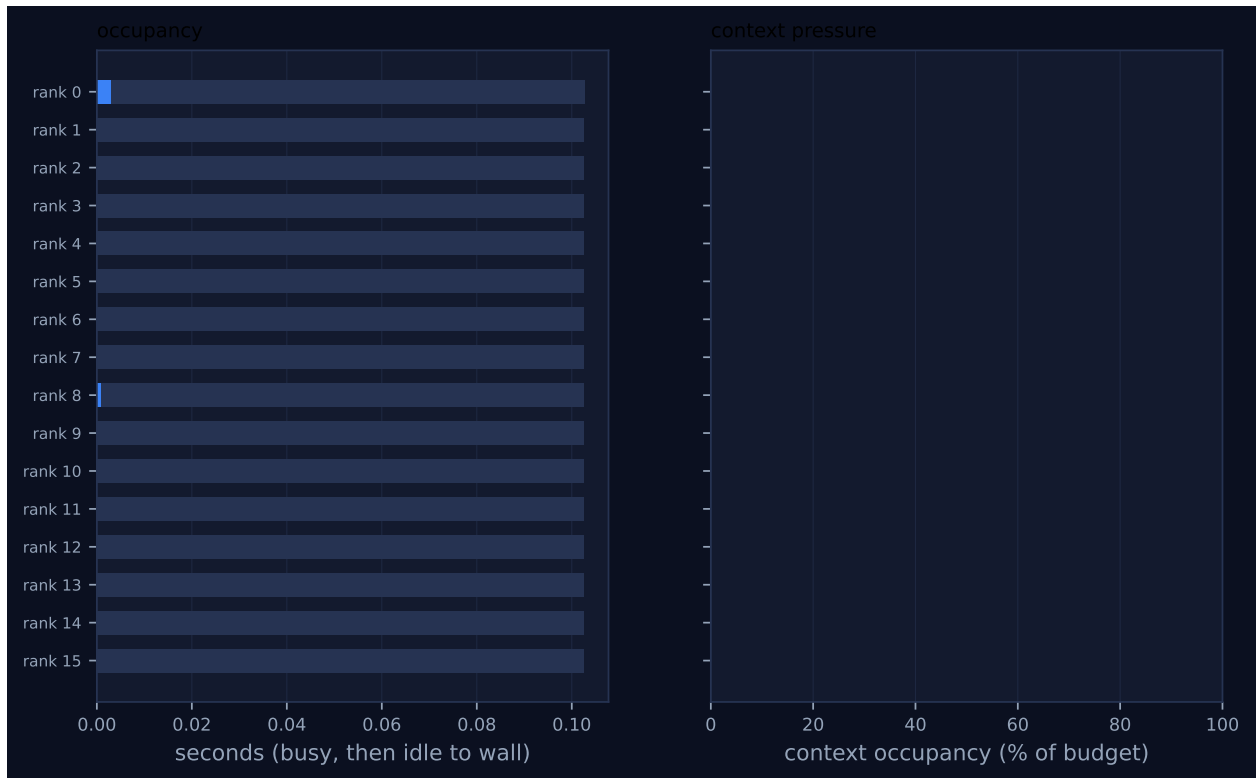


Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

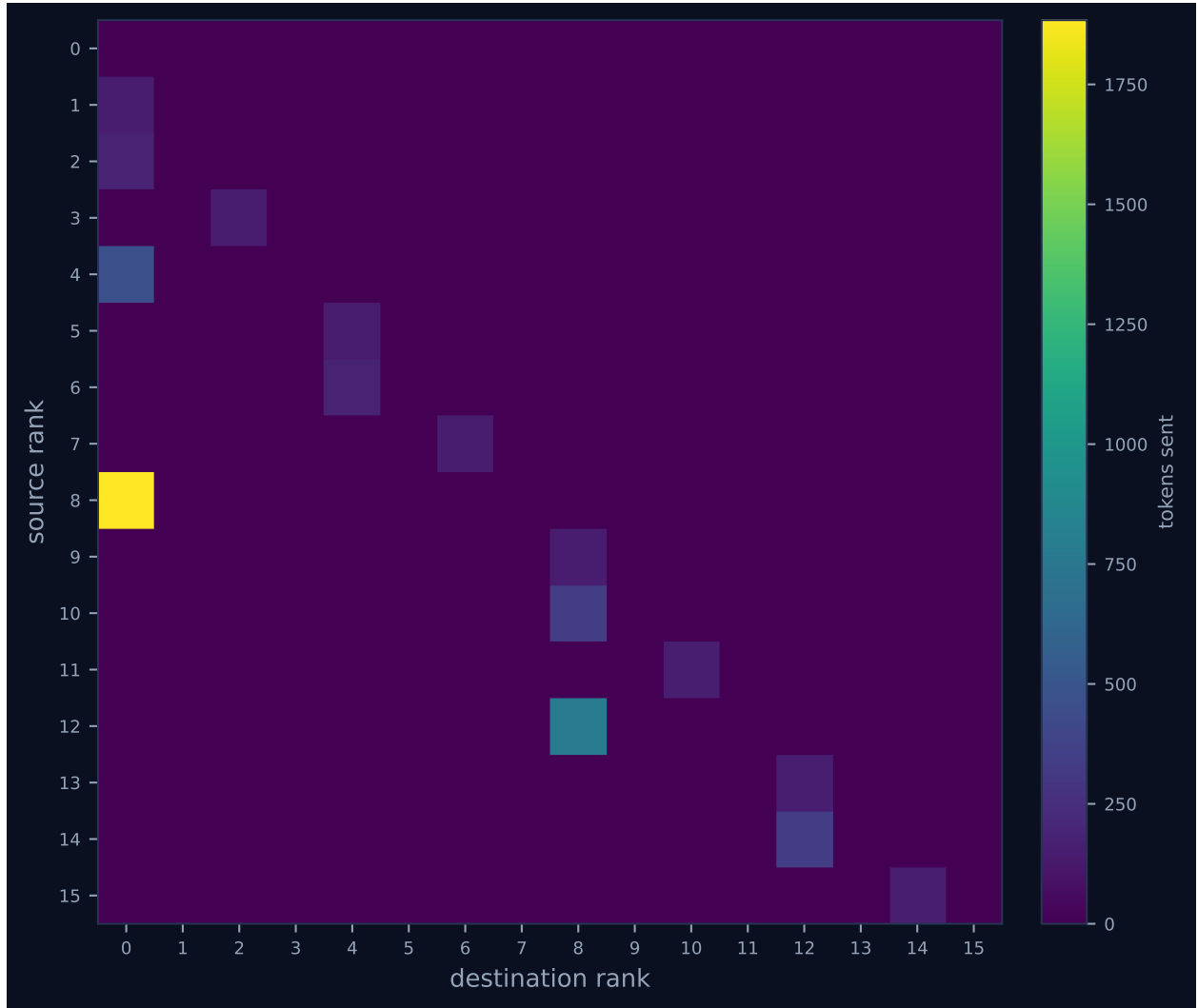


Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm’s shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

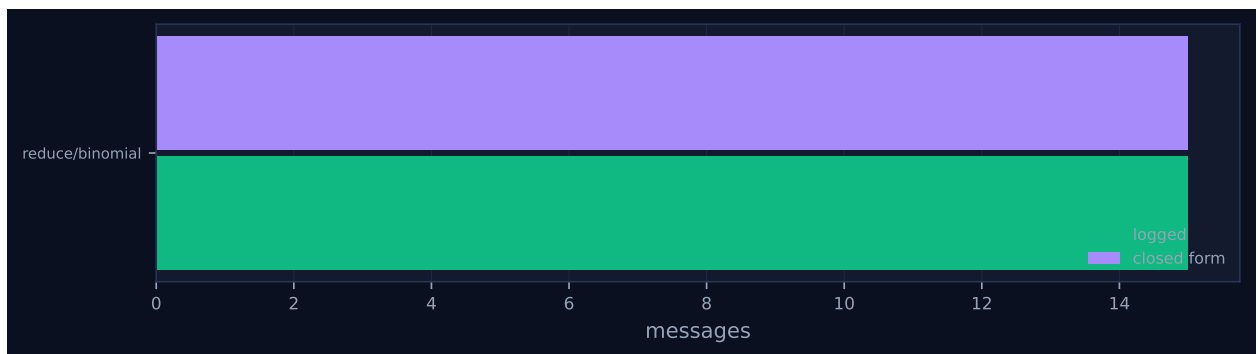


Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	3.0	2	4	0	4	0	0.0	finalized
1	0.00	0.0	0	0	1	0	153	0.0	finalized
2	0.00	0.0	0	1	1	1	188	0.0	finalized
3	0.00	0.0	0	0	1	0	153	0.0	finalized
4	0.00	0.0	0	2	1	2	458	0.0	finalized
5	0.00	0.0	0	0	1	0	153	0.0	finalized
6	0.00	0.0	0	1	1	1	188	0.0	finalized
7	0.00	0.0	0	0	1	0	153	0.0	finalized
8	0.00	2.2	1	3	1	3	1,884	0.0	finalized
9	0.00	0.0	0	0	1	0	153	0.0	finalized
10	0.00	0.0	0	1	1	1	341	0.0	finalized
11	0.00	0.0	0	0	1	0	156	0.0	finalized
12	0.00	0.0	0	2	1	2	783	0.0	finalized
13	0.00	0.0	0	0	1	0	156	0.0	finalized
14	0.00	0.0	0	1	1	1	341	0.0	finalized
15	0.00	0.0	0	0	1	0	156	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank’s own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
reduce	binomial	–	16	16	4	4	15	15	5,215	0.10	0.07	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 is a binomial tree drawn at millisecond resolution: the whole run is 0.103s, all fifteen merges are instants, and the visual structure is the collapsing pattern of active lanes rather than any duration. Eight ranks call the operator at depth 1 (the even ranks), four at depth 2 (0, 4, 8, 12), two at depth 3 (0, 8) and one at depth 4, which is the $\sum 2^{-i}$ staircase the algorithm’s `stride` loop produces, and the per-rank table shows it as the descending call counts 4, 0, 1, 0, 2, 0, 1, 0, 3, The event ordering confirms the levels interleave rather than synchronising: rank 0’s `merge:d2` fires before rank 14’s `merge:d1`, because a binomial reduce imposes no barrier between strides and a rank proceeds as soon as its own partner has arrived.

The occupancy row of the dashboard should be ignored, and the generated findings say why: these executors emit no `broker.claim` events, so achieved parallelism of $0.04\times$, the 99.8% idle fraction and the $12.00\times$ imbalance are artefacts of reconstructing busy time from an event stream that has none. What the timeline does show honestly is the token growth on the critical path, and it is the run’s most visible defect. The root’s four merges return 177, 439, 1,210 and 4,645 output tokens on prompts of 442, 514, 1,004 and 3,200, so each application roughly doubles the accumulator even though the underlying item count grows by at most a factor of two and the operator is nominally capped at 900 tokens. It doubles because the kernel copies its inputs verbatim into its output: the stored accumulator for the $8 \rightarrow 0$ message is a JSON string containing an escaped JSON string containing an escaped JSON string, three levels deep, at 1,884 tokens for 28 real identifiers. Total output across the run is 11,833 tokens for a fold over 64 four-token facts.

The communication matrix, Figure 4, is the tree’s signature: fifteen edges, 153 tokens on each leaf-to-parent link and 458 and 1,884 on the two edges into the root that carry accumulated subtrees, for 5,416 tokens in total. That is the cheapest traffic of the four cells at this p apart from `flat`, and the tree’s characteristic property — traffic per rank is constant while the accumulator carried on the last edge grows with p — is legible in the single 1,884-token edge from rank 8.

7 Interpretation

Everything structural is by construction and checks out, which is what a plumbing run is for. `fold_depth: 4` matches $\lceil \log_2 16 \rceil$ and is the shallowest of the three classical algorithms here, against `chain` and `flat` at 15; rounds are 4 against a predicted 4; messages are 15 against a predicted 15, and the finding that all checkable collectives agree with their closed form is the check passing. This cell is therefore the depth-4 reference point in the $p = 16$ series, and the series exists mainly to give the newly written `kary` branch something to be compared against at a process count where its geometry is clean.

What emerged, and what this document exists to record, is that the surrogate implements a different loss model from the one its docstring describes. The docstring claims each application independently drops each item with probability `drop_rate`, giving retention $(1 - r)^d$. The implementation collects candidate items by iterating `prompt.splitlines()` and keeping lines that match `FACT_ID`; but the merge prompt interpolates each input as `json.dumps(...)`, which emits no newlines, so a binary merge prompt yields exactly three matching lines: the instruction sentence containing the example identifier `[F-3-2]`, the entire left report, and the entire right report. The unit of loss is an accumulator, not a fact. Every stored accumulator in this run confirms it: `findings` has length 2 or 3, never 4 or more, and its first element is always the instruction sentence.

That has two measurable effects here. The first is a small upward bias in the metric. `fact_retention` counts identifiers appearing in the root’s result, and `[F-3-2]` appears in every accumulator in the run because the operator copies the prompt’s own example into its output; so rank 3 is credited with 0.25 retention — one of its four facts — while the blob for the $2 \rightarrow 0$ message shows its real report was discarded at rank 2’s first merge, leaving only rank 2’s four items and the phantom. The reported `n_retained: 45` therefore overstates transported facts by exactly one, and `n_hallucinated_ids: 0` is wrong in spirit if not in letter: the identifier was not invented by a model, it was copied from the harness’s own prompt, and the harness then counted it as evidence of retention. The second effect is more serious. The kernel seeds its generator with `seed + len(prompt)`, so two applications whose prompts have the same character length make identical draws. The eight depth-1 merges here divide into five with 1,681-character prompts (ranks 0–9, single-digit rank labels) and three

with 1,701 (ranks 10–15, two-digit labels). Replaying the kernel’s arithmetic on those prompts reproduces the run exactly: the five short prompts keep items 0 and 1 and drop item 2 — the right-hand input, which is always the odd-numbered partner — and the three long prompts keep all three. So ranks 1, 3, 5, 7, 9 lose everything and ranks 11, 13, 15 lose nothing, which is precisely the reported `per_rank_retention`, and $64 - 5 \times 4 + 1 = 45$ is precisely the reported retention. The positive `rank_position_correlation` of 0.3475 is not recency bias in a reducer; it is the fact that ranks above 9 have two-digit numerals.

The consequence for the campaign is that this cell’s retention number carries no weight in the depth argument, in either direction. Its siblings at this p score 0.875 (`flat`, depth 15), 0.6875 (`chain`, depth 15) and 0.6875 (`kary`, depth 2), so the ordering is not monotone in depth and the deepest algorithm wins; but that ordering is produced by how many applications happened to draw a repeated prompt length, not by any property of reduction. The real evidence about the depth hypothesis is in the broker campaigns, where all four algorithms agree exactly — at 1.000 under an advisory budget in `sat-mb-fidelity` and at 0.3854 under an enforced one in `inc-mb-fidelity-inc` — and it is the enforced budget, not the fold shape, that governs the outcome. The enforcement question does not arise in this run at all: at commit `c5e8e9e` the harness passed the module-level `MERGE_CONTRACT`, which has no `max_tokens` field, so the 900-token budget existed only as a sentence in the prompt. That is the advisory regime, and it is why a 4,645-token output at the root records `attempt: 1` and `contract_violations: 0`. Under a null bound, zero violations is uninformative rather than reassuring.

8 Threats to this reading

The executor is a surrogate, so nothing here is agent behaviour, and the reading above is about a Python function. That is the dominant threat and it applies to every sentence containing the word retention. It also means the run’s timing, cost and occupancy numbers describe the fabric: 0.103s of wall time is SQLite writes and event logging, and the \$0.2128 is what the modelled price list would have charged for 11,833 tokens the surrogate produced by string concatenation.

The mechanism claims are stronger than the retention claims, and I should be explicit about which is which. That the parsed items are three per prompt, that the first is the instruction sentence, and that accumulators nest verbatim are direct observations from the stored blobs. That the five depth-1 drops follow from equal prompt lengths is a replay of the kernel’s own arithmetic on prompts I rebuilt from `make_fact_report` with the run’s seed, and it predicts the observed `per_rank_retention` vector exactly, which is strong but is still a reconstruction rather than a logged fact — the `function` executor writes no rows to `agent_calls`, so this run’s fabric contains no prompts or results, only message payloads. The regime claim rests on reading `experiments/microbench.py` at the provenance commit rather than on a contract stored in the fabric, for the same reason; the behavioural corroboration is that outputs exceed 900 tokens fivefold with zero violations, which requires `max_tokens` to have been `None`.

The prompt-length coincidence is specific to this configuration and should not be generalised. It bites here because a binomial tree at $p = 16$ contains eight structurally identical depth-1 merges; `flat` and `chain` at the same p have prompts of fifteen distinct lengths, which is why they lost two and five ranks respectively by apparently unrelated draws. Nor should the defect be read as affecting the broker runs: they never call `_surrogate_merge`. The two defects I would report for fixing are the phantom identifier (the prompt’s example [F-3-2] should not be an identifier that `fact_retention` can score) and the line-based item extraction, which makes the surrogate’s loss

granularity a whole subtree; a third, milder one is the seeding, which should mix in the call index if the draws are meant to be independent.